Documento de Decisiones de Diseño

Actividad Colaborativa N.º 2 — Evolución del Sistema de Gestión Académica

Programa: Ingeniería de Sistemas
Asignatura: Programación
Estudiante: Brian David Bedoya Ramírez
Fecha: octubre de 2026

1. Introducción

La segunda actividad deja demostar el Sistema de Gestión Académica que se realizo en la primera parte de la actividad. La idea esta vez es implementar clases abstractas, composición y manejo de excepciones personalizadas sin remplazar el proyecto que se relizo de manera original. El sistema continua representando programas, asignaturas y personas, y en este caso administra grupos, horarios y matrículas.

2. Descripción general de la solución

Se conservan las clases ProgramaAcademico, Asignatura, Estudiante, Docente y Administrativo. Se adaptó Persona como clase abstracta y se agregaron GrupoAcademico, Horario y CatalogoAcademico. La lógica se separa en archivos que estan de modelos y se commprueba desde main.py. Los errores de negocio tienen clases propias en excepciones.py.

3. Abstracción de Persona y clases derivadas

Persona hereda de ABC y define realizar_actividad_principal() con el decorador @abstractmethod. No tiene sentido crear una Persona genérica, porque cada persona vinculada al sistema debe cumplir un rol. Esta decisión no deja que su instanciación directa y establece un contrato para las subclases. Estudiante describe su actividad de cursar un programa, Docente su labor de orientar clases y Administrativo su trabajo en una dependencia. Las tres implementaciones conservan la sobreescritura desarrollada en AC1.

4. Relaciones entre las entidades

GrupoAcademico recibe objetos existentes de Asignatura y Docente y mantiene referencias a ellos. Los estudiantes se agregan por medio de matricular_estudiante() y se guardan en un diccionario cuya clave es codigo_estudiantil, para identificar duplicados y permitir retiros. Cada asignatura mantiene su referencia al ProgramaAcademico. GrupoAcademico registra código, cupo, estudiantes y horario, y ofrece métodos de consulta de datos y matrículas.

5. Composición del horario

GrupoAcademico construye internamente un objeto Horario con día, hora inicial, hora final y salón. Se trata de composición porque el horario forma parte de la configuración del grupo y se crea como parte de

él. No corresponde a herencia: un grupo no es un horario, sino que tiene un horario. Horario valida que la hora inicial preceda a la final y proporciona una representación legible.

6. Excepciones personalizadas

EstudianteYaMatriculadoError se lanza en matricular_estudiante() cuando el código ya está registrado. GrupoLlenoError se lanza en el mismo método si no hay cupos. EstudianteNoMatriculadoError se lanza en retirar_estudiante() si no existe el código. AsignaturaNoEncontradaError y ProgramaNoEncontradoError se lanzan respectivamente desde buscar_asignatura() y buscar_programa() de CatalogoAcademico. El método intentar() de main.py captura estas excepciones, imprime mensajes comprensibles y permite continuar con los casos siguientes. Se evita confundir errores esperados del negocio con simples mensajes de condicionales.

7. Continuidad del polimorfismo

El polimorfismo de AC1 se mantiene porque Estudiante, Docente y Administrativo comparten el método realizar_actividad_principal() definido en Persona, pero cada objeto ejecuta su implementación específica. En main.py se recorre una colección heterogénea con un único ciclo, sin condicionales para distinguir los tipos.

8. Pruebas y verificaciones

main.py demu nos deja ver los once puntos solicitados: prohibición de instanciar Persona, creación de entidades y dos grupos, matrícula correcta y duplicada, cupo completo, retiro válido e inválido, búsqueda de asignatura y programa inexistentes y continuidad del programa. tests/test_ac2.py añade comprobaciones automatizadas de abstracción, polimorfismo, composición, validaciones y excepciones. La ejecución se realiza con python main.py y python -m unittest discover -s tests -v.

10. Conclusiones

La evolución permite modelar de forma más exacta concisa los roles universitarios, la oferta de grupos y el tramite de matrícula. La abstracción no deja crear personas sin un rol concreto; la composición representa correctamente el horario como parte del grupo; y las excepciones hacen explícitos los problemas del dominio sin interrumpir la ejecución. La organización modular conserva la base de la Actividad 1 y facilita futuras ampliaciones.